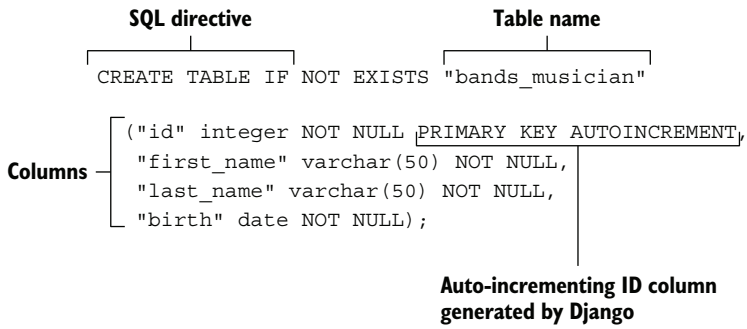


Figure 4.4 SQL used to create the table storing musicians info

empty (`NOT NULL`), it is the identifier other tables use to reference it (`PRIMARY KEY`), and it automatically increments when new rows get added (`AUTOINCREMENT`). The `first_name` and `last_name` columns are `varchar(50)`. A `varchar` is what SQL calls the `CharField`, and the `50` is the max length defined in your model. The `birth` field is of type `date`.

Not only is the `id` field not allowed to be empty (`NOT NULL`), but your three attribute fields are the same. This is the default behavior. You can control this through additional options to the field declaration (more on this later).

You can use the SQLite command-line tool to directly enter SQL queries. Your `Musician` class doesn't have any data yet, so there is nothing to see. You could write an SQL `INSERT` statement, but that isn't the ORM way of doing things. Leave the SQLite tool by running `.quit`, and learn the ORM way.

4.4 Model queries

Each `Model` class is both declarative and functional. Inheriting from `models.Model` and using the fields from the `models` package defines the structure of the tables that the ORM creates and manages for you. The base class also comes with a *query manager*, an interface for interacting with data in the database. You can create your own query manager, but for the vast majority of what you'll do with the ORM, the default query manager will be sufficient. The default, named *objects*, is attached to the base class.

The query manager is a class itself, and its methods get used for operations relating to your `Model`. You've created `Musician` and seen the resulting table in the database; now it is time to make some data. The `.create()` method on the query manager does exactly this. The arguments to `.create()` are the fields in your class that you wish to populate. As the `.create()` method is generic, you can't use positional arguments for the fields; you have to use keyword arguments for everything.

The other complication with arguments is which fields are required and which are optional. Because `.create()` is generic, the compiler can't tell if you're missing an argument. `Musician` doesn't have optional fields, so you need to provide keyword